



Universidade do Minho

RELATÓRIO

Sistemas Operativos

A109593 – Rui Pedro Longras Fernandes Ribeiro

A109507 – Carlos Daniel Gomes Martins

Índice

1. Introdução.....	1
2. Arquitetura do Sistema.....	3
2.1. Estrutura de Dados.....	3
2.2. Módulos.....	4
2.2.1. Controller.....	4
2.2.2. Runner.....	5
2.2.3. Common.....	5
3. Implementação por Fases.....	6
3.1. Fase 1.....	6
3.2. Fase 2.....	6
3.3. Fase 3.....	7
3.4. Fase 4.....	7
4. Testes e Resultados.....	7
5. Comparação de Políticas de Escalonamento.....	10
6. Conclusão.....	11

1. Introdução

Nos sistemas operativos modernos, a gestão de processos é um dos mecanismos mais importantes para garantir um bom funcionamento do Sistema. Quando múltiplos utilizadores submetem pedidos, é fundamental garantir que os recursos sejam partilhados de forma justa e eficiente.

O presente relatório descreve o desenvolvimento de um sistema de execução de comandos compostos por dois tipos de processos: o *controller* e o *runner*. O *controller* gere uma fila de pedidos submetidos por processos *runner* através de um *pipe*. De seguida, o *controller* decide quando o comando pode ser executado, com base no número de processos ativos e na política de escalonamento configurada. O programa foi projetado de maneira incremental em quatro fases distintas, desde a comunicação básica até ao suporte a *pipes* e redirecionamento de ficheiros.

Este documento apresenta a arquitetura do Sistema, as decisões tomadas para cada fase, descreve os testes e os seus resultados e a comparação das políticas de escalonamento e conclusões, incluindo as dificuldades encontradas e possíveis melhorias futuras.

2. Arquitetura do Sistema

O sistema baseia-se numa arquitetura cliente-servidor mediado por um canal de comunicação assíncrono. Os *runners* são os produtores de pedido, submetem comandos para execução e aguardam autorização, enquanto o *controller* desempenha a função de servidor central, recebendo os pedidos, gerindo a fila de espera e controlando o número de processos em execução simultânea.

2.1. Estrutura de Dados

O esquema central do sistema é a *Request*, definida em `common.h`, ou seja, a unidade de comunicação entre todos os processos. Cada vez que um *runner* interage com o *controller*, quer seja para submeter um comando, atualizar o seu estado ou solicitar uma consulta.

Imagem 1. Estrutura de dados

```
typedef struct {  
    int user_id;  
    int command_id;  
    char command[256];  
    char reply_pipe[64];  
    int status;  
    int operador_redirecionamento;  
} Request;
```

O campo status percorre os seguintes estados ao longo do ciclo de vida:

- Valor 0 → Pendente
- Valor 1 → Em execução
- Valor 2 → Concluído

- Valor 3 → Shutdown
- Valor 4 → Consulta

2.2. Módulos

2.2.1. Controller

O *controller* é o processo central do sistema, é responsável por gerir a fila de pedidos e controlar o paralelismo. Está configurado para iniciar com um limite máximo de processos em execução simultânea e com a política de escalonamento definida.

O ciclo principal lê continuamente estruturas *Request* do *pipe*. Os pedidos com status 4, são tratados imediatamente e não entram na fila. Para as restantes, determina-se uma atualização de estado de um pedido já existente ou de um novo pedido a ser inserido.

Quando um comando é concluído, o *controller* calcula o tempo decorrido e, posteriormente, independentemente do evento recebido, percorre o *array* à procura de comandos pendentes que possam ser lançados, respeitando sempre o limite de paralelismo definido.

O encerramento do *controller* apenas ocorre quando ("shutdown_pedido == 1 e running == 0"), garantindo que nenhum comando em execução é interrompido.

2.2.2. Runner

O *runner* corresponde ao processo cliente do Sistema, com recurso a ele, os utilizadores podem submeter comandos para execução, consultar o estado do sistema ou solicitar o encerramento do *controller*. O modo de funcionamento é determinado pelo primeiro argumento fornecido na linha de comandos.

No plano da comunicação, o *runner* cria um FIFO de resposta individual, envia uma estrutura *Request* pelo *pipe* principal e fica bloqueado à espera da resposta do

controller. A utilização de um *pipe* exclusivo por *runner* garante que cada processo recebe apenas a sua própria resposta.

Após receber autorização, o *runner* cria um processo filho. De seguida, o filho executa o comando, enquanto o processo pai aguarda a sua conclusão. No final, o *runner* notifica o *controller*, permitindo o registo no *log* e a libertação de um *slot* de paralelismo.

2.2.3. Common

O módulo *common* é a componente de suporte, não implementa lógica, mas fornece as primitivas de comunicação e a persistência utilizadas pelos outros dois módulos. É fundamental para isolar o protocolo de comunicação entre processos num único ficheiro, assim facilita a manutenção e a extensão futura.

As funções estão divididas em três grupos principais. As funções de comunicação do *runner* implementam os modos “-c” e “-s”, sendo responsáveis pela criação do *pipe* de resposta, envio do pedido e receção da resposta. As funções de comunicação do *controller* asseguram o envio de respostas aos *runners* através dos seus *pipes* individuais. Por fim, as funções de gestão de estado tratam da atualização do *array* de pedidos e do armazenamento permanente das execuções.

3. Implementação por Fases

A implementação do sistema seguiu uma metodologia incremental, estruturada em quatro fases de complexidade crescente. Cada fase corresponde a um conjunto de funcionalidades diferentes. Esta abordagem garantiu que os componentes da base do sistema, tais como as comunicações de inter-processos, a gestão do estado e controlo de paralelismo estivessem consolidadas antes das introduções de funcionalidades mais avançadas.

3.1. Fase 1

Na fase 1, a primeira decisão foi a implementação da estrutura de dados que serviria de base para todo o sistema. Optámos por uma única estrutura *Request* que contém toda a informação necessária para identificar o pedido, o utilizador e o canal de resposta, evitando a necessidade de múltiplas trocas de mensagens entre processos.

A segunda decisão passou por adotar um modelo de comunicação não uniforme, com um canal de entrada partilhado por todos os *runners* e um canal de saída individual para cada *runner*, assegurando o isolamento das respostas.

Por fim, a terceira decisão consistiu numa fase sem qualquer mecanismo de controlo, esta simplicidade permitiu validar o protocolo de forma isolada.

3.2. Fase 2

Na segunda fase, acrescentamos mecanismos de persistência e controlo. Tal como na fase anterior, foram implementadas três funcionalidades principais: o registo de execuções em ficheiro, a consulta do estado e, para finalizar, o encerramento controlado do *controller*.

3.3. Fase 3

A terceira fase, introduz o controlo de paralelismo e as políticas de escalonamento. O *controller* passou a receber dois argumentos na linha de comandos e a lógica foi reestruturada em torno desses parâmetros.

Começámos pela política FIFO que implementa um mecanismo de justiça reativo, em que o objetivo é percorrer todos os pedidos por ordem de chegada. De seguida, introduzimos a RANDOM visando imprevisibilidade deliberada. Por fim, acabámos com a FAIR, tendo como finalidade um método de justiça proativo.

3.4. Fase 4

A quarta fase, incorpora o suporte a comandos compostos no *runner*. Estes comandos foram implementados para tratar de cada tipo de operador de forma independente, fazendo o *parsing* do texto do comando antes de proceder à execução.

A principal decisão foi criar uma função dedicada, mantendo o main focado apenas à comunicação do *controller*. Optámos por redirecionar os descritores de ficheiro no processo filho e assim garantimos que o processo pai não fosse afetado. No caso das pipelines, escolhemos criar um processo filho por segmento do comando, ligados através de um pipe Unix, fechando todos os extremos no processo pai após o fork para evitar bloqueios por EOF.

4. Testes e Resultados

Nesta secção apresentam-se os resultados obtidos durante os testes realizados ao sistema. Os resultados completos podem ser consultados nos ficheiros “avaliacao_politicas.txt” e “avaliacao_politicas2.txt”, onde são comparadas diferentes políticas de escalonamento e diferentes níveis de paralelismo. Em todos os testes foram registados o tempo total de execução do sistema, o tempo individual de cada comando e o tempo total de execução associado a cada utilizador.

Imagem 2. Resumo dos resultados de “avaliacao_politicas.txt”

```
Cenário
- U1: 5x sleep 3 | U2: 5x sleep 1
- U3: 5x sleep 2 | U4: 3x sleep 1 + 2x sleep 3

Paralelismo 2

FIFO
- total: 21.12 s
- U1: 12.04 s | U2: 7.04 s | U3: 16.02 s | U4: 17.04 s

RANDOM
- total: 21.14 s
- U1: 13.08 s | U2: 17.07 s | U3: 18.08 s | U4: 16.02 s

FAIR
- total: 21.10 s
- U1: 18.08 s | U2: 17.04 s | U3: 18.02 s | U4: 16.04 s

Paralelismo 4

FIFO
- total: 12.09 s
- U1: 6.03 s | U2: 6.01 s | U3: 8.05 s | U4: 8.00 s

RANDOM
- total: 13.09 s
- U1: 6.08 s | U2: 8.07 s | U3: 7.05 s | U4: 9.01 s

FAIR
- total: 11.07 s
- U1: 10.05 s | U2: 7.03 s | U3: 8.02 s | U4: 7.00 s
```

Nos testes presentes no ficheiro “avaliacao_politicas.txt” foi utilizado um cenário equilibrado entre utilizadores. O utilizador 1 submeteu cinco comandos sleep 3, o utilizador 2 submeteu cinco comandos sleep 1, o utilizador 3 submeteu cinco comandos sleep 2 e o utilizador 4 submeteu três comandos sleep 1 e dois comandos sleep 3. Este cenário permitiu comparar diretamente o comportamento das diferentes políticas de escalonamento.

Com paralelismo 2, o tempo total de execução foi semelhante entre as políticas, situando-se próximo dos 21 segundos. No entanto, verificaram-se diferenças na distribuição dos tempos de execução entre utilizadores. Com paralelismo 4 observou-se uma redução significativa do tempo total de execução, destacando-se a política FAIR com o melhor resultado global, próximo dos 11 segundos.

Imagem 3. Resumo dos resultados de “avaliacao_politicas2.txt”

```
Cenário
- U1: 8x sleep 3 | U2: 8x sleep 3
- U3: 2x sleep 1 | U4: 2x sleep 1

Paralelismo 2

FIFO
- total: 28.11 s
- U1: 21.05 s | U2: 27.07 s | U3: 22.03 s | U4: 24.07 s

RANDOM
- total: 27.09 s
- U1: 26.07 s | U2: 26.05 s | U3: 22.06 s | U4: 19.03 s

FAIR
- total: 27.10 s
- U1: 26.08 s | U2: 26.06 s | U3: 6.04 s | U4: 7.03 s

Paralelismo 4

FIFO
- total: 15.11 s
- U1: 11.03 s | U2: 14.07 s | U3: 4.03 s | U4: 4.01 s

RANDOM
- total: 16.08 s
- U1: 15.06 s | U2: 13.03 s | U3: 3.01 s | U4: 8.05 s

FAIR
- total: 15.08 s
- U1: 14.06 s | U2: 14.04 s | U3: 4.03 s | U4: 4.03 s
```

No ficheiro "avaliacao_politicas2.txt" foi utilizado um cenário mais extremo, onde os utilizadores 1 e 2 submeteram oito comandos sleep 3, enquanto os utilizadores 3 e 4 submeteram apenas dois comandos sleep 1. Além disso, os utilizadores com maior carga de trabalho submeteram os seus comandos primeiro, permitindo analisar o impacto das políticas de escalonamento no tempo de espera dos restantes utilizadores. Com paralelismo 2, a política FIFO provocou tempos de espera elevados para os utilizadores 3 e 4, enquanto a política FAIR reduziu drasticamente esses tempos para aproximadamente 6 e 7 segundos. Com paralelismo 4 verificou-se novamente uma redução significativa do tempo total de execução em todas as políticas testadas, demonstrando a importância do paralelismo e da política de escalonamento no desempenho global do sistema.

5. Comparação de Políticas de Escalonamento

A análise dos resultados obtidos permite observar diferenças claras entre as políticas de escalonamento implementadas. A política FIFO executa os processos estritamente por ordem de chegada, o que resulta num comportamento previsível, mas pode aumentar significativamente o tempo de espera dos utilizadores que submetem poucos comandos após utilizadores com cargas mais pesadas. Este comportamento foi especialmente visível no ficheiro “avaliacao_politicas2.txt”, onde os utilizadores com menos processos ficaram bloqueados atrás de vários comandos longos submetidos anteriormente.

A política RANDOM apresentou resultados intermédios, distribuindo os processos de forma menos previsível devido à seleção aleatória do próximo comando a executar. Em alguns cenários conseguiu reduzir o tempo de espera de determinados utilizadores, porém, os resultados variaram consoante a ordem aleatória escolhida durante a execução dos testes.

Por outro lado, a política FAIR demonstrou um comportamento mais equilibrado entre utilizadores, distribuindo os recursos de forma mais justa e reduzindo significativamente o tempo de espera dos utilizadores com menos processos submetidos. Esta diferença tornou-se particularmente evidente no cenário do ficheiro “avaliacao_politicas2.txt”, onde os utilizadores 3 e 4 concluíram os seus processos muito mais rapidamente em comparação com a política FIFO.

Os resultados demonstram também que o aumento do paralelismo teve um impacto significativo no desempenho global do sistema. Em praticamente todos os cenários testados, a utilização de paralelismo 4 permitiu reduzir substancialmente o tempo total de execução quando comparado com paralelismo 2, melhorando o aproveitamento dos recursos disponíveis e reduzindo o tempo médio de espera dos utilizadores

6. Conclusão

O desenvolvimento deste projeto permitiu consolidar os conhecimentos adquiridos na Unidade Curricular de Sistemas Operativos, nomeadamente no que diz respeito à comunicação inter-processos, ao escalonamento e ao controlo de paralelismo. A abordagem incremental adotada revelou-se fundamental, permitindo validar cada componente de maneira isolada antes de avançar para funcionalidades mais complexas.

As maiores dificuldades surgiram na Fase 3, na implementação da justiça entre utilizadores, e na Fase 4, na gestão dos descritores de ficheiro durante a execução de pipelines. Todavia, conseguimos perceber que os resultados dos testes confirmaram que a política FAIR é a mais equilibrada em cenários com utilizadores de carga assimétrica, enquanto a política FIFO pode ser a melhor escolha em casos de carga relativamente simétrica. Podemos assim concluir, que a escolha da melhor política depende sempre da carga a que será submetida.

Em suma, este projeto permitiu aplicar na prática conceitos fundamentais de sistemas operativos, num sistema funcional e testado. Como melhorias futuras, destacámos a substituição do *array* fixo por uma estrutura dinâmica, o suporte a *pipelines* com mais do que um operador e a refactorização do *controller* para eliminar a duplicação de código entre as três políticas.